TraitHorizon: Scalable Exploration of Large Image-Feature Paired Datasets

11 November 2025

1. Summary

Collections of Objects of Interest (OOIs)—such as cells, tubules, or tissue patches—paired with high-dimensional, computationally derived feature vectors (e.g., morphology, texture, or deep-learned embeddings) are increasingly being generated in image-based biomedical research. These OOI-feature datasets are routinely explored to detect outliers for quality control, uncover patterns, and generate biological insights. However, as these datasets grow in size and complexity, researchers face several bottlenecks: repeated manual creation of static visualizations for each subpopulation, difficulties in efficiently plotting large numbers of high-dimensional feature vectors, and limited tooling for tracing individual feature values back to their source objects. To address these challenges, we developed TraitHorizon, a browser-based visualization platform for interactive exploration of large OOI–feature pair datasets. TraitHorizon integrates three synchronized visualization components: (a) a parallel coordinates plot for feature-vector–level exploration, (b) dynamic violin plots for per-feature distribution analysis, and (c) a tabular data grid linking each feature vector to its corresponding object. Its interactive interface enables real-time, scalable visualization of datasets containing hundreds of thousands of OOI-feature pairs. We demonstrate TraitHorizon's utility through a case study involving quality control of a digital pathology dataset comprising 260,201 segmented tubules in kidney biopsies, each characterized by 99 features, illustrating how the platform facilitates rapid, interpretable, and reproducible data interrogation.

TraitHorizon is publicly available for use and modification at https://traithorizon.com/.

2. Statement of Need

Research in image-based fields (e.g., computational pathology) often aims to discover trends in Objects of Interest (OOIs), associated with diagnosis, prognosis, and therapy response. Objects can consist of individual items (e.g., cells), higher-

order structures (e.g., tubules or infiltration patterns), or even small image patches. From these OOIs, sets of numerical features, termed feature vectors, are often extracted to encode their attributes. Features can range from simple (e.g., area, stain intensity, texture) to more complex, hand-crafted (e.g., aspect ratio of peritubular capillaries [1]), or even deep learning-derived descriptors [1–5]. These features can then be examined to assess their associations with object phenotypes—such as biological signals (e.g., disease state) or image quality factors (e.g., blurriness)—or how they positively/negatively correlate with one another.

Such analysis often involve generating static, non-interactive, visualizations, such as a parallel coordinates plots [6] or scatter plots with UMAP [7] or t-SNE [8]. Aided by first-order summary statistics (e.g., mean, standard deviation), these visualizations are inspected to uncover structure and outliers within the feature distribution. Multiple data-driven subpopulations often emerge from this initial exploration, prompting repeated generation of visualizations and summary metrics for each dataset. Quantitative and qualitative patterns uncovered within subpopulations can, in turn, guide comparative analyses—for example, assessing how a given feature pattern differs between "diseased" and "normal" groups.

However, to date, this investigatory workflow remains time-intensive due to three main limiting factors:

1. **Lack of connection between an object's image, feature values, and cohort-level visualization.** Analysts often discover outliers or clusters within cohort-level visualizations, prompting the need to trace them back to the associated images for visual inspection. Static plots can, at best, display only a modest number of images, forcing analysts to manually map plotted points to objects, incurring significant time costs. The ability to instantly view the object associated with a plotted feature vector is essential for rapid validation, interpretation, and insight generation.
2. **Exploring each subpopulation incurs high time cost.** Biomedical imaging datasets often include heterogeneous subpopulations—such as tissue types, disease subtypes, or experimental conditions—that warrant the generation of separate visualizations for detailed inspection. Manually plotting each subpopulation becomes inefficient in repetitive workflows, while programmatic (e.g., loop-based) plot generation requires subpopulations of interest to be identified and well-defined *a priori*. Dynamic filtering and responsive plots are necessary for users to iteratively explore subpopulations without incurring substantial time costs from repeated plot generation and review. Furthermore, the conditions/filters used to identify them should be easy to document, save, and reapply.
3. **Latency while rendering large datasets disrupts analysis.** OOI-feature datasets can easily reach hundreds of thousands of data points. When subpopulations are not yet clearly defined, a "global" view of the entire dataset is required before smaller subpopulations can be explored. Browser-based tools not designed for plotting at this scale can suffer from

heavy memory utilization, latency, and even unresponsiveness/browser crashes while rendering such data visualizations. A capable visualization tool must leverage rendering techniques that remain efficient and within browser limitations as the number of objects scales.

These limiting factors motivate the development of interactive data visualization tools that can dynamically generate visualizations as the user explores their data. However, existing tools do not fully address all three limiting factors simultaneously. TensorBoard Projector [9] and HoloViews [10] support interactive plotting of large multivariate datasets but do not provide for image viewing. Conversely, DendroMap [11] facilitates qualitative exploration of large sets of images but does not link back to their feature vectors. These tools also do not natively support dynamic filtering by feature value. HistoQC [12] provides image-linked plots and metrics, but it is tailored specifically to quality control use cases in digital pathology and thus does not support arbitrary user-defined features vectors or images.

To address these gaps, we introduce TraitHorizon, a browser-based application for interactive exploration of images alongside their feature vectors. TraitHorizon (a) directly links plotted data points to their source object images, enabling rapid validation and interpretation, (b) supports dynamic, multi-dimensional filtering with support for saving and loading filter configurations, and (c) efficiently renders hundreds of thousands of data points in-browser. By doing so, TraitHorizon supports efficient exploratory analysis in object-based research workflows.

3. Implementation

TraitHorizon is a Flask-based application [13], making it ideal for collaborative environments when hosted over a network. The front end is built with JavaScript, HTML5, and CSS, and leverages visualization libraries such as SlickGrid [14], Parallel Coordinates (parcoords [15]), and D3.js [16] to power its interactive dashboard. Its command-line interface ingests a single directory of object image files (e.g., .png, .jpg, .svg, or any other browser-supported format [17]) and a tab-separated values (TSV) file, with each row containing an image file path followed by a user-determined number of tab-separated feature columns. As such, any tool outputting tabular data is compatible with TraitHorizon's simple input format (e.g., HistoQC [12] and CohortFinder [18]). TraitHorizon supports integer, float, scientific notation, and categorical features. Clickable URLs are also supported by an optional column titled "url".

TraitHorizon is designed to meet the demands of datasets containing hundreds of thousands of objects. To minimize browser lag during plotting, five application features were implemented:

1. Non-blocking progressive rendering occurs in the parallel coordinates plot, completing in seconds to minutes depending on dataset size. Importantly, the user interface remains responsive during this rendering period, allowing

users to continue interacting with the parallel coordinates plot and other functional components without straining browser responsiveness.
2. Parallel coordinate filter configurations and associated filtered object IDs can be exported and imported in a lean JSON format, without requiring a re-render of the parallel coordinates plot.
3. The data grid supports both pagination and alternatively "infinite scrolling". This web design technique enables new content to be dynamically loaded, replacing old content as the user scrolls down and creating the illusion of a never-ending page within a fixed memory footprint.
4. Object images are loaded on-demand to limit browser memory, network utilization, and CPU usage.
5. Violin plots are also computed on-demand when the user hovers over a parallel axis, avoiding CPU usage associated with refreshing violin plots each time the parallel coordinates plot is updated.

These rendering techniques allow TraitHorizon to operate smoothly within the constraints of modern web browsers and modest consumer-grade hardware, regardless of dataset size.

4. Use Case – Interactive Exploration of Tubular Pathomic Features via TraitHorizon

To illustrate TraitHorizon's functionality, we provide the following detailed use case from a recent digital pathology image-based biomarker study [2,19]. In our use case, we focused on data exploration of 260,201 segmented tubules [20] and their associated features (for a total of 99 features per tubule) as computed from 254 Periodic Acid-Schiff (PAS) stained kidney biopsies from the Nephrotic Syndrome Study Network (NEPTUNE) [21,22] and Cure Glomerulonephropathy (CureGN) [23] consortia. The goal was to support quality control and uncover tubular morphological patterns along a pathway from normal to tubular atrophy [20,24].

4.1. Overview and Dataset Preparation

Before running TraitHorizon, users must prepare a TSV file containing the following columns:

1. **"filename" (required)** – Specifies the base filename for each instance image, without the folder path (e.g., image1.png).
2. **Features (required)** – A tab separated row containing all features associated with each instance. Our dataset included 99 tubular pathomic features capturing morphological and topological characteristics of the basement membrane, epithelium, nuclei and lumen [2]. In our pre-TraitHorizon analysis, non-negative matrix factorization [25,26] (rank = 14) followed by UMAP on these features revealed 1,667 tubules forming outlier clusters. These were flagged in a separate "Needs QC" column.

3. **"url" (optional)** – Allows linking each object to external resources such as secondary visualizations, related analyses, or web-based viewers. In our use case, these URLs open the corresponding tubule in the HistomicsUI viewer [27] within the Digital Slide Archive (DSA) [28], which displays the tubule's location within the original tissue slide for added histologic context.

4.2. Running TraitHorizon

The folder containing all image files should be provided to TraitHorizon's command-line interface via the assets_path argument. All images must reside in this directory, which TraitHorizon uses to locate and display them in the user interface. Similarly, the path to the TSV file should be specified using the `tsv_path` argument.

After starting the TraitHorizon server, a local URL (e.g., http://localhost:5000) will appear in the terminal. Opening this URL in a web browser launches the TraitHorizon user interface Figure 1, allowing users to interactively explore images and their associated features.

4.3. Filtering Based on Features(s)

5. Acknowledgements

6. References

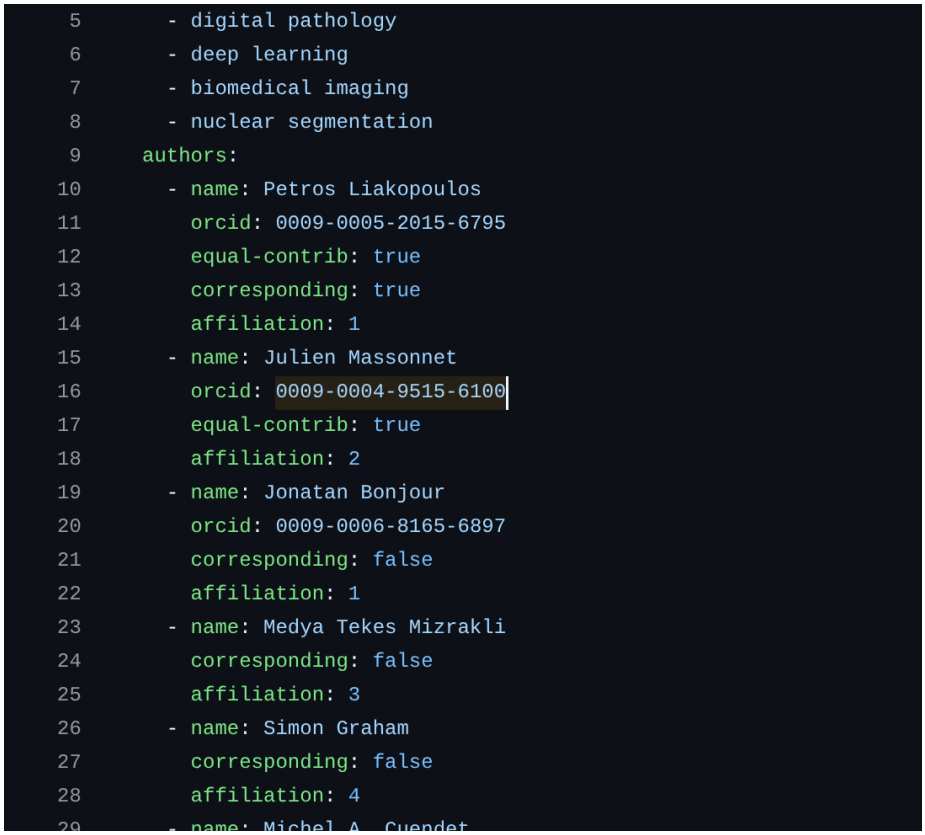

Figure 1: TraitHorizon web application interface. (A) The interactive parallel coordinates plot displays 99 pathomic features from 260,201 tubules, wherein each blue line represents one tubule feature vector. (B) A violin plot shows the value distribution of a selected feature when hovered over. (C) The data grid presents each segmented tubular image, with the original PAS-stained tubule on the left and the segmentation overlay on the right (green: tubular basement membrane; white: tubular nuclei; black: tubular lumen; red: tubular epithelium). Associated feature data loaded from the TSV (e.g., filename and corresponding feature values) are also displayed. (D) The bottom status bar shows the number of instances currently visible. (E) The drop zone and export button allow users to save filter configurations and filtered row IDs. TraitHorizon allows users to customize which columns are included or excluded in the parallel coordinates plot. Here, the URL and filename columns were excluded from the parallel coordinates plot but are shown in the data grid.